

1. Práctica 3: Entrada / Salida programada - UART TX

1.1. Objetivos

- Comprender más en profundidad el concepto de "periférico" dentro de un procesador.
- Aprender a programar un periférico.
- Interactuar con el periférico desde fuera de la placa.

1.2. Fundamentos teóricos

1.2.1. UART

La comunicación **UART** (*Universal Asynchronous Receiver-Transmitter*) es un protocolo de comunicación serie asíncrono utilizado ampliamente en sistemas embebidos. Al ser asíncrono, no requiere de una señal de reloj compartida entre el emisor y el receptor; en su lugar, ambos dispositivos deben estar configurados previamente a la misma velocidad de transmisión (*baud rate*).

Estructura de la Trama UART:

En estado de reposo (*Idle*), la línea de transmisión se mantiene en nivel lógico alto (**1**). Una trama estándar de datos está compuesta por los siguientes elementos:

1. **Bit de Inicio (*Start Bit*):** Transición de nivel alto (**1**) a nivel bajo (**0**) durante un período de bit. Notifica al receptor que una nueva trama está comenzando.
2. **Longitud de Palabra (*Data Bits*):** Bloque de datos útil, transmitido habitualmente empezando por el bit menos significativo (*LSB first*). Suele configurarse entre 5 y 9 bits, siendo 8 bits el valor más común.
3. **Bit de Paridad (*Parity Bit, opcional*):** Bit añadido al final de los datos para la detección elemental de errores:
 - **Paridad Par (*Even*):** Se ajusta a 1 o 0 para que el número total de unos en la palabra de datos (incluyendo el bit de paridad) sea par.
 - **Paridad Impar (*Odd*):** Se ajusta para que el número total de unos sea impar.
4. **Bits de Parada (*Stop Bits*):** Uno o varios bits en nivel lógico alto (**1**) al final de la trama que garantizan un tiempo de resincronización mínimo para la línea antes de la siguiente transmisión.

Diagrama de Tiempos Típico:

En la Figura 1 se muestra la representación gráfica de una trama serie de 8 bits de datos con paridad y 1 bit de stop:

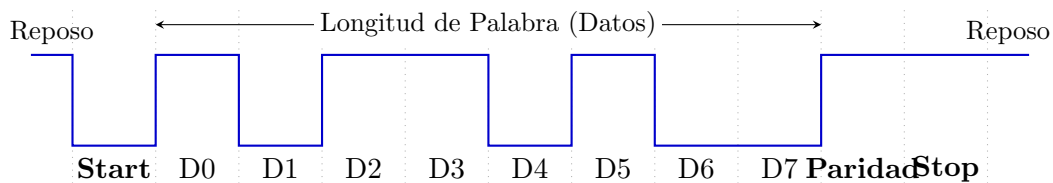


Figura 1: Cronograma de una trama de datos UART de 8 bits con paridad y 1 bit de parada.

1.2.2. UART en el STM32F723IE

En el procesador de la placa, se disponen de varios periféricos de UART diferentes, numerados como USARTx, siendo la x el número de UART. La UART dispone de varios registros de control, otros de configuración, y finalmente registros para la recepción/envío de datos. Todos ellos se describen en [rm0431 27.8](#). Las direcciones base de los periféricos USARTx se encuentran en [rm0431 1.6.2](#). Cómo hacer la transmisión en sí se describe en [rm0431 27.5.2](#).

¿Ahora, qué USARTx utilizamos? en [um2140 5.13](#) nos dice que la USART6 está conectada directamente a la salida desde ST-Link (El depurador que venimos utilizando hasta ahora para conectar con la placa). Por tanto, deberemos configurar USART6. Los pines donde se conecta (que habrá que activar) se pueden ver en [ds11853 Table 10](#). En concreto, vemos que son el pin PC6 para USART6_TX y PC7 para USART6_RX. (De momento solo usamos el TX para transmitir). Para que por ese pin salga la UART (Y no el valor genérico que ponemos en el puerto de salida del GPIO), deberemos configurar la función alternativa 8 como indica [ds11853 Table 12](#), que conecta la USART6 al puerto C.

1.3. Desarrollo de la práctica

Para el desarrollo de esta práctica hay que extender el BSP introduciendo las funciones y definiciones necesarias de la UART, y posteriormente controlarla para emitir datos hacia el ordenador. La documentación necesaria puede verse en [rm0431 27](#). Los pasos a seguir son los siguientes:

1. Según [ds11853 Table 10](#), el pin PC6 es quien saca al exterior la UART. Configúralo en modo de función alternativa en [MODER](#), tipo push-pull en [OTYPER](#), pull-up en [PUPDR](#), velocidad alta en [OSPEEDR](#), y configura la función alternativa en [AFR](#) para que sea la 8 (como indica [ds11853 Table 12](#)). No te olvides de activar el puerto C a través de [AHB1ENR](#).

```
1 RCC_AHB1ENR->bits.GPIOCEN = 1;
2 GPIO_MODE_SET(GPIOC, 6, GPIO_MODE_ALT);
3 //... completar
```

2. Ahora le toca el turno a la USART6 como nos indica [um2140 5.13](#). Actívala desde [APB2ENR](#) lo primero.

```
1 //en bsp.h
2 #define RCC_APB2EN_BASE_ADDR 0x40023844
3 //Structure for Reset Control peripheral register APB1
4 typedef union {
5     volatile uint32_t reg;
6
7     // 2. Access individual bits or bit-groups for the register above
```

```

8     struct {
9         volatile uint32_t TIM1EN          : 1;  // Bit 0:
10        volatile uint32_t TIM8EN          : 1;  // Bit 1:
11        volatile uint32_t RES1             : 2;  // Bit 2-3:
12        //... completar
13    } bits;
14 } RCC_APB2R_TypeDef;
15 #define RCC_APB2ENR          ((RCC_APB2R_TypeDef *) RCC_APB2EN_BASE_ADDR)
16
17
18 //en main.c
19 RCC_APB2ENR->bits.USART6EN = 1;          //enable UART clock

```

3. Configura la UART desde los registros **CR1** y **CR2**. Activa el periférico, configura la longitud de palabra a 8 bits, 1 bit de stop y sin paridad, pone por último el oversampling a 16.
4. Configura la velocidad de la UART desde **BRR**. Tendrás que ponerle el valor del reloj del sistema (16MHz) entre la velocidad de la uart (115200).
5. Por último, activa el modo de transmisión desde **CR1**.

```

1  // UART STOP BITS
2  typedef enum {
3      UART_STOP_ONEBIT = 0x00,
4      UART_STOP_HALFBIT = 0x01,
5      UART_STOP_TWOBIT  = 0x02,
6      UART_STOP_ONEHALFBIT = 0x03
7  } UART_Stop_t;
8  #define UART_STOP_MASK 0b11
9  #define UART_STOP_SHIFT 12
10
11 static inline void UART_STOPBIT_SET(volatile USART_TypeDef *USARTx,
12     UART_Stop_t stopbits) {
13     //... completar configurar el registro CR2 con los bits de stop
14 }
15
16 // UART ENABLE
17 typedef enum {
18     UART_DISABLE = 0x00,
19     UART_ENABLE = 0x01
20 } UART_Enable_t;
21 #define UART_ENABLE_MASK 0b1
22 #define UART_ENABLE_SHIFT 0
23
24 static inline void UART_ENABLE_SET(volatile USART_TypeDef *USARTx,
25     UART_Enable_t enable) {
26     //... completar configurar el registro CR1 con el bit de enable
27 }
28
29 // UART WORD LENGTH
30 typedef enum {
31     UART_WORD_8B = 0x00,
32     UART_WORD_9B = 0x01,
33     UART_WORD_7B = 0x02
34 } UART_WordLength_t;
35 #define UART_WORD_MASK 0b1
36 #define UART_WORD_SHIFT 12
37
38 static inline void UART_WORDLENGTH_SET(volatile USART_TypeDef *USARTx,
39     UART_WordLength_t length) {
40     //... completar escribir longitud en registro CR1

```

```

38 }
39
40 //... completar todas estas otras funciones auxiliares
41 static inline void UART_PARITY_SET(volatile USART_TypeDef *USARTx,
    UART_Parity_t parity);
42 static inline void UART_OVERSAMPLING_SET(volatile USART_TypeDef *USARTx,
    UART_Oversampling_t over);
43 static inline void UART_TX_MODE_SET(volatile USART_TypeDef *USARTx,
    UART_TX_Mode_t mode);

```

6. Cuando quieras transmitir un byte, espera primero a que quede libre el transmisor (aparece un 1 en el bit TC del registro **ISR**). A continuación escribe el dato en el registro **TDR**, la UART lo transmitirá solo.

```

1 //... en bsp.h
2 static inline void UART_TransmitByte(volatile USART_TypeDef *USARTx,
    uint8_t data)
3 {
4     while (!(USARTx->ISR & (1U << 7U))); //Wait until we can push data (TXE
        bit enabled)
5     USARTx->TDR = data; //push data (this clears TXE bit)
6 }
7
8
9 //... en main.c
10 UART_TransmitByte(USART6, 'X');
11 UART_TransmitByte(USART6, 'Y');
12 //...

```

Para ver ahora los mensajes desde el ordenador, utilizaremos la utilidad **picocom** (o cualquier otra terminal de vuestro gusto). Desde la máquina virtual, abrir una terminal y ejecutar el comando:

```

1 picocom /dev/ttyACM0 -b 115200 -d 8 -y n -p 1

```

NOTA: El nombre del puerto puede no ser necesariamente **ttyACM0**, aunque en linux al menos es el valor por defecto. En Windows, por ejemplo, suele llamarse **COMx**, siendo x un número arbitrario.

NOTA: Si no está instalada la utilidad **picocom**, se puede instalar con:

```

1 > sudo apt-get install picocom

```

1.3.1. Envío de cadenas de texto por UART

Aunque mola mucho programar en *bare-metal*, también es útil partir de librerías ya implementadas para facilitar la vida a los programadores. Podemos ampliar la funcionalidad de nuestra UART para imprimir mensajes de depuración (así en próximas prácticas podremos depurar vía mensajes, en ocasiones más cómodo e intuitivo que la depuración con *breakpoints*). Incluye las siguientes funciones y prueba a depurar tu código con mensajes.

En **bsp.h** tendrás que añadir una función para escribir cadenas completas:

```

1 void UART_WriteString(USART_TypeDef *usart, const char *str) {
2     while (*str) {

```

```

3     UART_TransmitByte(usart, *str++);
4 }
5 }

```

También tendrás que añadir la función de depuración en sí con las librerías necesarias auxiliares:

```

1  #include <stdint.h>
2  #include <stdlib.h>
3  #include <stdio.h>
4  #include <string.h>
5  #include <stdarg.h>
6
7  void Debug_Printf(const char *fmt, ...)
8  {
9      char buf[256];
10     va_list args;
11     va_start(args, fmt);
12     vsnprintf(buf, sizeof(buf), fmt, args);
13     va_end(args);
14
15     UART_WriteString(USART6, buf);
16 }

```

Y listo! Puedes llamar a la función `Debug_Printf` con el texto con formato que quieras, para imprimir los valores de variables, por ejemplo, o simplemente colocar mensajes de depuración por la UART.

1.4. Para hacer más

Investiga las interrupciones de la UART para evitar tener que hacer esperas activas que consuman recursos de CPU.